The Computational Power of Distributed Shared-Memory Models with Bounded-Size Registers

Carole Delporte-Gallet
carole.delporte@irif.fr
IRIF, University Paris Cité, CNRS
Paris, France

Hugues Fauconnier
hugues.fauconnier@irif.fr
IRIF, University Paris Cité, CNRS
Paris, France

Pierre Fraigniaud
pierre.fraigniaud@irif.fr
IRIF, University Paris Cité, CNRS
Paris, France

Sergio Rajsbaum
rajsbaum@matem.unam.mx
IRIF and Instituto de Matemàticas, UNAM
Mexico City, Mexico

Corentin Travers
corentin.travers@lis-lab.fr
LIS, University Aix-Marseille, CNRS
Marseille, France

ABSTRACT

The celebrated *Asynchronous Computability Theorem* of Herlihy and Shavit (JACM 1999) provided a topological characterization of the tasks that are wait-free solvable by processes communicating through writing and reading shared registers. This characterization assumes the use of *full-information* protocols, in which each time a process writes in the shared memory, it communicates everything it learned since the beginning of the execution. Thus, each register in the shared memory is of *unbounded* size. Whether unbounded size registers are unavoidable for the model of computation to be *universal* is the central question studied in this paper. More generally, when at most t out of n processes can crash, is the model with bounded size registers universal?

We show that when $1 \leq t < n/2$ registers of size $O(t)$ bits are sufficient to solve any task that is solvable by a full-information protocol. The situation is radically different when $n/2 < t < n$, which includes the wait-free case. We show that bounded registers are *not* universal. Specifically, we show that, for any bound $f : \mathbb{N} \to \mathbb{N}$, and for any $n > 2$, there exists a task that cannot be solved by n processes communicating via registers of size $f(n)$ bits, but is solvable by n processes using unbounded registers. The case $n = 2$, $t = 1$, where 1-resilient computing and wait-free computing coincide, does not display the same behavior. We show that any task solvable by 2 processes in the wait-free shared-memory model with unbounded-size registers can be solved by 2 processes in the wait-free shared-memory model with constant size registers.

Finally, we consider the *iterated* shared memory model, which has played a crucial role since Herlihy and Shavit's characterization. We show that 1-bit registers (per iteration) are sufficient for solving wait-free any task that is solvable by a full-information wait-free protocol. Therefore, while the wait-free read/write shared-memory model is known to be computationally equivalent to the iterated model, this equivalence only holds if registers are unbounded.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '24, June 17–21, 2024, Nantes, France
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0668-4/24/06
https://doi.org/10.1145/3662158.3662789

CCS CONCEPTS

• **Theory of computation → Concurrent algorithms**; **Distributed computing models**.

KEYWORDS

Shared memory, Wait-freedom, t-resiliency, Approximate agreement, Space complexity

ACM Reference Format:
Carole Delporte-Gallet, Hugues Fauconnier, Pierre Fraigniaud, Sergio Rajsbaum, and Corentin Travers. 2024. The Computational Power of Distributed Shared-Memory Models with Bounded-Size Registers. In *ACM Symposium on Principles of Distributed Computing (PODC '24), June 17–21, 2024, Nantes, France.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3662158.3662789

1 INTRODUCTION

1.1 Objective

In sequential computing, universal models such as the Turing machine were identified from the early days of modern computer science. In distributed computing, the power of a model may however depend on the reliability of its components, and on other assumptions. We consider in this paper the standard *asynchronous read/write shared memory* model, where t out of n processes may crash, and the shared memory consists of n *single-writer/multi-reader* registers (SWMR), one per process. Even for this most basic model the problem is not fully resolved, as we discuss next.

A characterization of the tasks that are solvable when $t = 1$ is known [9]. In the other extreme, for $t = n - 1$, the celebrated *asynchronous computability theorem* [23] provided a characterization of the tasks that are wait-free solvable in the read/write model. The characterization was finally extended to the general t-resilient case [35], and other models, for an overview see [22]. In a distributed task, each process is initially given a private input and is required to provide an output. The task specifies the set of possible input and output, and what are the legal outputs for each assignment of inputs to the processes. All these results assume *full-information* protocols, in which each time a processes writes in its register, it

writes everything it learned since the beginning of the execution, and each process remembers all the values it has read from the registers of the other processes. Therefore, a fundamental assumption about the model of computation is that each register in the shared memory is of unbounded size.

The question addressed in this paper is the following. Are there distributed tasks that can be solved in the asynchronous read/write shared memory model assuming unbounded registers, but that cannot be solved in the same shared-memory model whenever the size of the registers is bounded? That is, we question the existence of a *universal* model with bounded-size registers. We answer this question for the wait-free model, $t = n - 1$, and more generally, for the t-resilient version of the model, for $t \in \{1, \ldots, n-1\}$.

1.2 Our Results

In a nutshell, we show that bounded size registers yield a universal model for read/write shared-memory computing only when less than half of the processes may fail. More specifically, we first show that wait-free computing with bounded registers is not universal, for $n > 2$ processes. This actually holds more generally, for the t-resilient model whenever $t > \frac{n}{2}$. Moreover, the result holds in a strong sense, that is, even if one allows the size of the registers to depend on the number n of processes in the system.

Theorem 1.1. *For any function $f : \mathbb{N} \to \mathbb{N}$, for any $n > 2$, and for any t with $\frac{n}{2} < t < n$, there exists a task that is solvable by n processes in the t-resilient shared-memory model with unbounded-size registers, but that cannot be solved by n processes in the t-resilient shared-memory model with registers of at most $f(n)$ bits.*

We stress that the task whose existence is established in this theorem has binary inputs, i.e., each process starts with an input value in $\{0, 1\}$. The unsolvability of the task is therefore not due to large-size inputs that the processes may need to write to small registers.

Theorem 1.1 holds for every $n > 2$. The case $n = 2$, and where 1-resilient computing and wait-free computing coincide, actually displays a radically different behavior.

Theorem 1.2. *Any task solvable by 2 processes in the wait-free shared-memory model with unbounded-size registers can be solved by 2 processes in the wait-free shared-memory model with registers of 3 bits.*

This theorem holds for arbitrary tasks, even those with arbitrarily large inputs, by assuming that each of the two processes has access to a special write-once register in which it can write its input, which can be read by the other process. These registers are solely used for exchanging inputs, and cannot be used for coordination between the processes, which is the core of the distributed computation.

We show that the threshold in Theorem 1.1 regarding the maximum number t of failures is essentially the best one can expect. Indeed, for $t < \frac{n}{2}$, registers of bounded size are universal, as shown by the following result.

Theorem 1.3. *There exists $\beta \geq 1$ such that, for any $n \geq 2$, and any t with $1 \leq t < \frac{n}{2}$, every task solvable by n processes in the t-resilient shared-memory model with unbounded-size registers can be solved by n processes in the t-resilient shared-memory model with registers of $\beta\, t$ bits.*

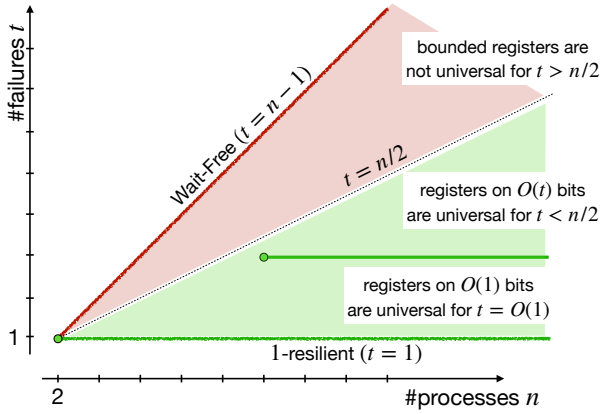


Figure 1: *Summary of our results. For $t > \frac{n}{2}$, the t-resilient model with bounded registers is not universal, even if the bound on the size of the registers may depend on the number of processes (Theorem 1.1). For $t < \frac{n}{2}$, the t-resilient model with registers on $O(t)$ bits is universal (Theorem 1.3). Therefore, for $t = O(1)$, the t-resilient model with constant-size registers is universal. Moreover, for the case $n = 2, t = 1$, 1-bit registers are universal (Theorem 1.2).*

Note that the constant β does not depend on the task, neither on the number of processes. In particular, every task solvable by n processes in the 1-resilient shared-memory model with unbounded-size registers can be solved by n processes in the 1-resilient shared-memory model with constant size registers. Figure 1 summarizes all these results.

Notice that neither Theorem 1.1 nor Theorem 1.3 encompasses de case when exactly half of the processes may fail (n is even and $t = \frac{n}{2}$). This case if left for future work.

We then consider the *iterated* version of the wait-free model, which is an abstraction that has been used in most papers since the asynchronous computability theorem [23] was established, because it greatly simplifies topological arguments [22]. Here, instead of a single array M of n registers (one register per process), the shared-memory consists of an infinite sequence $(M_r)_{r \geq 1}$ of such arrays. The computation is organized in rounds. At every round $r \geq 1$, the processes communicate by writing once, and then reading once the registers in the rth memory M_r. Several variants of iterated models have been studied, some even more powerful than wait-free [26–28], but the general idea is that the restriction of accessing only once each memory M_r, and the inability of re-reading a "lower" memory M_s for $s < r$, can be done without loss of generality, as far as task solvability is concerned. Iterated models assume unbounded registers: in each array M_r, the size of each register is unbounded. Specifically, in the wait-free model, a task is solvable if and only if it is solvable in the corresponding iterated model [11, 19], both using unbounded registers. The equivalence relies on full-information protocols, in which, for each r, each process writes to its registers of M_r what is has read in $M_1, \ldots, M_{r-1}$ We show that, in fact, bounded size registers suffice in the iterated model.

Theorem 1.4. *For any $n \geq 2$, any task solvable wait-free by n processes in the iterated immediate snapshot model with unbounded*

registers is solvable wait-free in the iterated immediate snapshot model with 1-bit registers (at each M_r).

Therefore, while the wait-free read/write shared-memory model is known to be computationally equivalent to its wait-free iterated version, this equivalence holds only if registers are unbounded, by Theorem 1.1.

Finally, we conclude the paper by using insights of our iterated model analysis. We show in Theorem 1.2 that 3-bit registers are universal for two processes. The proof relies on a novel binary approximate agreement algorithm, that uses registers of 1 bit. However, using 1-bit registers comes at the price of an exponential slowdown compared to approximate agreement algorithms using unbounded registers. The step complexity is $O(1/\epsilon)$ when using 1 bit registers. Nevertheless, we show that this overhead can actually be eliminated using slightly larger, but still constant-size registers.

Theorem 1.5. *There exists a wait-free ϵ-agreement protocol for two processes with step complexity $O(\log \frac{1}{\epsilon})$ using registers of size 6.*

1.3 Related Work

The fundamental paper by Fischer, Lynch, and Paterson [17] showed that Turing computability theory is different than computability in distributed systems, by proving that the consensus task is not solvable in an asynchronous message passing system even if only one process may fail by crashing, even if each process is an infinite state machine. Biran, Moran and Zaks [9] extended the result into a full characterization of the tasks that are solvable in this model of computation. Later, it was shown by Attiya, Bar-Noy and Dolev [4] that the message passing and shared memory models are equivalent with respect to the tasks that they can solve, when the number of processes that can crash t is $t < n/2$.

Biran, Moran and Zaks [9] showed that any 1-resilient solvable task can be solved by a form of approximate agreement on a graph, defined by the possible outputs of the task. It is "approximate" in the sense that processes decide on the same vertex or on adjacent vertices of the graph. In Section 4 we show how this approach also applies to the case of two processes communicating with constant size registers. For $t < n/2$, we prove the universality of registers of size $O(t)$ in Section 5, transforming the algorithm with unbounded registers to a message passing algorithm using the above simulation [4], then reducing the communication links between processes, as long as the network is $(t+1)$-connected, and finally relying on the alternating bit protocol (see [8, 29]).

The approximate agreement problem where processes start with real valued inputs in a range $[a, b]$ was introduced by Dolev, Lynch, Pinter, Stark, and Weihl [16], and since then it has been thoroughly studied, providing solutions in various models, both synchronous and asynchronous, even when processes can exhibit arbitrary, Byzantine failures, e.g. [1]. Closer to our work, Hoest and Shavit [25] considered the problem in a shared-memory iterated model of computation, proving matching upper and lower bounds of $\log_d(b-a)/\epsilon$ on number of steps taken by a process, where $d = 3$ for two processes, and $d = 2$ for three or more processes. A similar result was proved in message passing, dynamic graph models [18].

In contrast, when a majority of the processes may fail, restricting the size of the registers strictly limits the degree of approximation that can be achieved. In Section 3, we show that there exists a sufficiently small ϵ, such that approximate agreement cannot be solved by n processes in the t-resilient shared-memory model with registers of $O(n)$ bits.

Generalizing the task solvability characterization [9] from $t = 1$ to $t = n - 1$ was done by Herlihy and Shavit [23] using combinatorial topology, essentially going from 1-dimensional graphs needed to model two processes executions, to $(n+1)$-dimensional simplicial complexes needed to model n process executions. In particular, an n-dimensional form of approximate agreement task played the analogue of the approximate agreement graphs task used in [9]. An extensive literature has investigated the solvability of particular tasks, such as consensus, approximate agreement [16], set agreement [34], and renaming [12]. For an overview of results in this area, and why it is intimately related to topology, see [22]. Most of the work uses iterated models, but a few have studied non-iterated models e.g. [6].

Analyzing the topology of the protocol complex representing the executions of an algorithm, proved to be much easier in an iterated model, which has the same task computability power [11]. Dynamic network models where synchronous processes that do not crash communicate by sending messages that may be lost are closely related to iterated shared memory models [3, 18, 20]. It was shown that, for two processes, it is possible to solve any wait-free solvable task using 1-bit messages, without incurring any cost in the optimal number of communication rounds [14]. In Section 6 we generalize this result for any $n \geq 2$, showing that any task solvable wait-free by n processes in the IIS model with unbounded registers is solvable wait-free in the IIS model with 1-bit registers (per round). For this, we use the snapshot implementation of [10].

For the case of algorithms with constant size registers, the input registers allow processes to communicate their inputs for free, to separate the concerns about the size of the inputs from the question about reducing the costs of the full information protocol under asynchrony and failures. For example, bounds on approximate agreement often depend both on ϵ and on the magnitude or the spread of the inputs e.g. [5, 21, 30, 36]. Instead, we separate the aspects related to the number of possible inputs, and the actual approximate agreement computation, by assuming instead inputs $\{0, 1\}$, and then use a solution to this binary version, to solve any task. In a sense, communicating the input is an information theoretic concern, and indeed, the problem has been thoroughly studied from this perspective, e.g. Orlitsky [31], and more recently [32], and this is the case also in the wait-free setting [14]. Indeed, Schenk [36] (see also [37, Theorem 3.11] for more details) shows that any two processes protocol for approximate agreement communicating through b-bits MWMR registers must use a number of registers that grows as ϵ gets smaller, an apparent contradiction with our solution using two 1-bit registers. The reason is precisely that Schenk assumes inputs in $[0, 1]$, while we assume only two possible inputs. Attiya and Ellen [5] focus on multi-dimensional version of approximate agreement (and MWMR registers), but in the one-dimensional case, achieve an approximate agreement algorithm whose step complexity is $O(\min\{\log n(\log n + \log(S/\epsilon)), \log(M/\epsilon)\})$, where S is the spread $b - a$ and M is their magnitude, the largest of the absolute values of the inputs. In contrast, our lower bounds for more than 2 processes showing the impossibility of solving wait-free ϵ-agreement with constant (or bounded) registers size holds even

if processes have only two possible inputs. Schenk considers also the wait-free case of $n > 2$, and proves upper and lower bounds of $\Theta(\log(s/\epsilon))$ single-bit MWMR registers, for real-valued inputs of magnitude at most s, while we show a solution with registers whose size does not depend on ϵ, for $t < n/2$, and furthermore, in the case of a majority of failures we show that there must be a dependence on ϵ, even in the case of only two possible inputs.

1.4 Paper Organization

In Section 2, we formally describe our model, and we recall basic facts from the distributed computing framework that will be used for establishing our own results. Then, in Sections 3-6 we provide the reader with an overview about the main arguments used for proving our results. The complete proofs of the four Theorems 1.1-1.4 can be found in the full version of the paper [13].

Section 7 revisits Theorem 1.2, and demonstrates that, by slightly increasing the size of the registers, from 1 bit to $O(1)$ bits, the simulation used in the proof of Theorem 1.2 can be speeded up exponentially, and running at essentially the same time as the algorithms using unbounded registers. The details and complete proof can be found in the full version [13]. Finally, Section 8 concludes the paper.

2 MODEL AND BASIC FACTS

We use a standard model of computation that is briefly summarized below, together with reminders of classical results in distributed computing that will be useful in the paper. More details can be found in standard textbooks [7, 33].

Computing Model. We consider in this paper the *asynchronous read/write shared memory* model, consisting of $n \geq 2$ deterministic processes, $p_1, \ldots p_n$, modelled as (possibly infinite) state machines, communicating by reading and writing to a shared memory. The shared memory consists of n atomic single-writer multi-reader (SWMR) registers $R_1, \ldots, R_n$, one per process. A write operation performed by a process in a register R is $R.\text{write}(v)$, meaning that the value v is written in register R. Similarly, $R.\text{read}()$ returns the value v currently stored in register R. An atomic operation performed by a process in the shared memory is called a *step*. The processes are *asynchronous*, and may fail by *crashing*.

Tasks and Algorithms. A *task* is the basic form of distributed computing problem, the analogous of a function for Turing machines, studied since [9]. It is defined by a set of input *configurations* of the n-processes system, a set of output configurations, and a mapping specifying, for every input configuration, the set of output configurations that are legal with respect to this input. An algorithm solves a given task if each of the n processes can start with any private input value taken from the input domain of the task, communicate with the other processes, and eventually decide with a private output value such that the collection of output values produced by the processes is consistent with the collection of input values with respect to the input-output specification of the task.

More formally, a task for n processes is defined as a triplet $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$ where $\mathcal{I}$ and $\mathcal{O}$ are finite sets of input and output configurations (usually defined as simplicial complexes), and $\Delta : \mathcal{I} \to 2^{\mathcal{O}}$ maps each input configuration $\sigma \in \mathcal{I}$ to a set of output configurations $\Delta(\sigma) \subseteq \mathcal{O}$ that are legal with respect to σ. An algorithm $\mathcal{A}$ solves a task Π if whenever each of the n processes is initially given a private input such that the sets of inputs define a configuration $\sigma \in \mathcal{I}$, any execution of $\mathcal{A}$ results in the (non crashing) processes deciding output values forming a configuration $\tau \in \mathcal{O}$ that is legal for σ according to the input-output specification Δ, i.e., $\tau \in \Delta(\sigma)$.

Executions. An *execution* (of an algorithm) is a sequence of *steps*, i.e., a sequence of read or write operations in the shared memory performed by any of the processes. A *solo* execution is an execution in which only one of the n processes take steps, all the others being either crashed or very slow. A process that does not crash in an execution, is said to be *correct* in that execution.

Resiliency. The model has different variants, parametrized by the maximum number of crashes which may occur during any execution (of an algorithm). For every $t \in \{1, \ldots, n-1\}$, the *t-resilient* model assumes that at most t processes can crash during any execution. The case $t = n - 1$ deserves special attention, and is referred to as the *wait-free* model.

Consensus. Consensus is arguably the most important task in distributed computability. Every process i starts with a value $x_i \in I$ taken from some finite set I, and every correct process p_i must decide an output value y_i such that (1) the value y_i is equal to the input value x_j of some p_j, and (2) the output values decided by the correct processes are identical. The instance of consensus in which $I = \{0, 1\}$ is *binary consensus*. As mentioned previously, solving consensus is impossible, even in the stronger model considered in the paper (i.e., the 1-resilient model), a result originally proved in a message passing system [17], but can be extended to shared memory [7].

Lemma 2.1. *For every $n \geq 2$, binary consensus cannot be solved by n processes in the 1-resilient shared-memory model.*

Approximate Agreement. Approximate agreement [15] is a relaxation of binary consensus, parametrized by a "degree of agreement" $\epsilon > 0$. When the domain is discrete $\{0, 1\}$, for $\epsilon \in (0, 1]$, the binary ϵ-agreement task is the following. Each process starts with an input value in $\{0, 1\}$. It must produce an output value in the interval $[0, 1]$, such that (1) if all processes start with the same value $x \in \{0, 1\}$, then they must decide that value x, and (2) otherwise they can output any values in $[0, 1]$ as long as all these values are at most ϵ apart from each other. To be formalized as a task, a discretized version of approximate agreement has been introduced (see [22]), in which $\epsilon = 1/k$ for some integer $k \geq 1$, and it is required that all the output values be of the form m/k, for some $m \in \{0, \ldots, k\}$. As opposed to consensus, ϵ-agreement is solvable, for every $\epsilon > 0$, even in the weakest variant considered in the paper, i.e., the wait-free model e.g. [22, 25].

Lemma 2.2. *For every $\epsilon > 0$ and every $n \geq 2$, ϵ-agreement can be solved by n processes in the wait-free shared-memory model.*

Size of the Registers. This paper is mostly interested in the *size*, b, of the shared registers $R_1, \ldots, R_n$. There is a subtlety regarding the role of the registers, because they serve two purposes. One is that processes may need to communicate each other inputs to solve a task. The other is that even when processes already succeeded

in communicating each other inputs, they may need to continue exchanging information, to be able to solve the task; this is clearly exposed by our impossibility results, which hold for tasks with constant inputs, actually inputs in $\{0, 1\}$.

For handling tasks with arbitrarily large inputs in the context of our positive results using registers of bounded size (whether it be constant or a function of n), we assume one additional SWMR *write-once* register I_i per process p_i, $i \in \{1, \ldots, n\}$. In this case, the first operation of each process p_i is writing its input x_i (if any) in its register I_i. This register can be read at will by the other processes, but p_i never writes again to its register I_i. That is, the registers $I_1, \ldots, I_n$ are just to store input values. Coordination is based only on the b-bits registers $R_1, \ldots, R_n$. We stress that our impossibility results hold even for binary inputs.

Snapshots and Immediate Snapshots. A *snapshot object* [2] allows any process i to read all the registers of the other processes as if this would occur atomically. The instruction snapshot() returns all the values stored in the registers in the memory, as an n-entry vector. An *immediate* snapshot [10] is an even more structured object enabling any process p_i to write in its register, and then snapshot the memory, as a single atomic action, as if the snapshot occurred "immediately" after the write, yet still wait-free implementable [10]. As opposed to all the previously mentioned communication instructions (read, write, and snapshot), several immediate snapshots may occur concurrently.

Lemma 2.3. *For every $n \geq 2$, an immediate snapshot can be implemented by n processes in the wait-free read/write shared-memory model.*

The Iterated Model. The *iterated immediate snapshot* model (IIS) is similar to the asynchronous read/write shared memory model, except that the processes have access to an infinite sequence of shared memories $M_1, M_2, \ldots$. Each memory M_r is composed of n SWMR registers $R_{r,i}$, $i = 1, \ldots, n$, where p_i is the only process allowed to write into $R_{r,i}$. For every $r \geq 1$, a process performs a single immediate snapshot operation on M_r, and proceeds to round $r + 1$. By definition, the iterated model can be simulated by the non-iterated model (assuming unbounded registers). The interesting fact is that both models are computationally equivalent [11].

Lemma 2.4. *For every task Π and for every $n \geq 2$, Π is solvable by n processes in the wait-free shared-memory model if and only if Π is solvable by n processes in the wait-free IIS model.*

3 NON UNIVERSALITY WHEN A MAJORITY OF PROCESSES MAY FAIL

The proof of Theorem 1.1 is by demonstrating that, given $f : \mathbb{N} \to \mathbb{N}$, $n > 2$, and $t \in (\frac{n}{2}, n-1]$, there exists $\epsilon > 0$ sufficiently small such that ϵ-agreement cannot be solved by n processes in the t-resilient shared-memory model with registers of at most $f(n)$ bits. This will be sufficient for establishing the theorem since, thanks to Lemma 2.2, for every $\epsilon > 0$, ϵ-agreement is solvable by n processes in the wait-free model, and therefore in the stronger t-resilient model as well, whenever using unbounded-size registers.

To get an intuition of why ϵ-agreement cannot be solved by $n > 2$ processes t-resilient whenever ϵ is too small, let us consider the simplest case of wait-free computing (i.e., up to $n - 1$ processes can crash). Intuitively, registers of size at most $s = f(n)$ bits causes the following issue. If two processes p_1 and p_2 run solo, and reach agreement, their outputs are at most ϵ apart from each other. However, a register of size s bits can take at most 2^s different values, and therefore, if $\epsilon \ll \frac{1}{2^s}$, they cannot accurately store their outputs in the shared registers. As a consequence, if a third process p_3 does not take any step before p_1 and p_2 terminate, it cannot determine its output with sufficient precision for guaranteeing that it will be at most ϵ apart from the outputs of p_1 and p_2.

Note that this proof heavily uses the wait-free assumption. For instance, if $n \geq 4$ and at most $n - 2$ processes could fail instead of at most $n - 1$ as in the wait-free case, then, in the scenario depicted in the proof, the two processes p_1 and p_2 could wait for each other, and coordinate to solve consensus. Instead, in the wait-free setting, these processes are bounded to output values that are ϵ apart from each other in some executions, precisely because none of the two processes can ever wait for the other process, at any point in time during all executions in which only p_1 and p_2 take steps.

For general case with $n > 3$ and $n > t > n/2$, the proof is by contradiction. Our proof technique shares some similarities with the one uses in[36] to establish lower bounds on the number of constant-size register for approximate agreement (in the case where inputs are any real in the range $[0, 1]$.). Let us assume that, for every $\epsilon > 0$, there exists a t-resilient algorithm A_ϵ solving ϵ-agreement for $n < 2t$ processes, using registers of size $f(n)$ bits. It is sufficient to consider $\epsilon = 1/k$ for some odd integer $k > 1$. For such an ϵ, the sets of possible inputs and outputs are respectively $\mathcal{I} = \{0, 1\}$ and $\mathcal{O} = \{0, \frac{1}{k}, \ldots, \frac{k-1}{k}, 1\}$. For every $\ell \in \{0, \ldots, k-1\}$, let

$$O_\ell = \left\{ \frac{\ell}{k}, \frac{\ell+1}{k} \right\}.$$

That is, O_ℓ is a valid legal set of outputs in an execution in which the input set is $\{0, 1\}$.

Claim 3.1. *For every $\ell \in \{0, \ldots, k-1\}$, there exists an execution of $\mathcal{A}_\epsilon$ satisfying that (1) only processes $p_1, \ldots, p_{n-t+1}$ take steps, and (2) at least $n - t$ processes decide, and the set of outputs is equal to O_ℓ.*

Proof. Let us assume, for the purpose of contradiction, that there exists $\ell \in \{0, \ldots, k-1\}$ such that, for every execution of $\mathcal{A}_\epsilon$ in which only the processes $p_1, \ldots, p_{n-t+1}$ take steps, the set of outputs is not O_ℓ. To establish a contradiction, we are going to show that, under this assumption, processes $p_1, \ldots, p_{n-t+1}$ could use algorithm $\mathcal{A}_\epsilon$ to solve 1-resilient binary consensus, contradicting Lemma 2.1. Note that since $n \geq 3$, and $t > n/2$, we have always $t > 1$, and thus $n - t + 1 \leq n - 1$.

Let us assume that every process p_i, $i \in \{1, \ldots, n - t + 1\}$ is given an input for binary consensus (i.e., 0 or 1). Process p_i first runs algorithm $\mathcal{A}_\epsilon$. Since we are interested in solving binary consensus in the 1-resilient model, we consider only the executions in which at most one process in $\{p_1, \ldots, p_{n-t+1}\}$ fails. Hence, in the corresponding execution of $\mathcal{A}_\epsilon$, we have that $t - 1$ processes fail initially, and at most 1 additional process fails, perhaps after having taken some steps. Since $\mathcal{A}_\epsilon$ tolerates t failures, each process p_i, $i \in \{1, \ldots, n - t + 1\}$ that does not fail will eventually produce

an output, say d_i. To solve binary consensus, process i outputs

$$\text{out}(p_i) = \begin{cases} 0 & \text{if } d_i \leq \frac{\ell}{k} \\ 1 & \text{otherwise.} \end{cases}$$

In this way, binary consensus is solved. Indeed, if the only input is 0 (resp., 1), then the only value returns by $\mathcal{A}_\epsilon$ is 0 (resp., 1), by the validity property of ϵ-agreement. Moreover, let D be the set of output values returned by $\mathcal{A}_\epsilon$. As $\mathcal{A}_\epsilon$ solves ϵ-agreement, we have

$$D \subseteq O_{\ell'} = \left\{ \frac{\ell'}{k}, \frac{\ell'+1}{k} \right\}$$

for some $\ell' \in \{0, \ldots, k-1\} \setminus \{\ell\}$. If D is a singleton, then all processes that decide have outputted the same value, as desired. If $D = O_{\ell'}$, then, as $\ell' \neq \ell$, we have

$$\frac{\ell'}{k} < \frac{\ell'+1}{k} \leq \frac{\ell}{k} \text{ or } \frac{\ell+1}{k} \leq \frac{\ell'}{k} < \frac{\ell'+1}{k}.$$

Every process that decides thus output 0 in the first case, and 1 in the second case, from which agreement follows. This contradicts the impossibility of 1-resilient binary consensus (cf. Lemma 2.1), and completes the proof of Claim 3.1. □

To complete the proof of Theorem 1.1, we now set

$$k = 2\left(2^{f(n)}\right)^{n-t+1} + 1.$$

For each $\ell \in \{0, \ldots, k-1\}$, let $\mathcal{E}_\ell$ be a finite execution of $\mathcal{A}_\epsilon$ in which only processes in $\{p_1, \ldots, p_{n-t+1}\}$ take steps, and the set of outputs is O_ℓ. Such an execution exists by Claim 3.1. Let w_ℓ be the binary word formed by concatenating the contents of the registers $R_1, \ldots, R_{n-t+1}$ immediately after each process $p_1, \ldots, p_{n-t+1}$ has decided. Since each register has size at most $f(n)$ bits, the set W of possible such words satisfies

$$|W| = \left(2^{f(n)}\right)^{n-t+1}.$$

Let us consider the set of $\frac{k-1}{2}+1$ output sets $\{O_0, O_2, \ldots, O_{k-1}\}$. These sets are mutually exclusive in the sense that if, in some execution of $\mathcal{A}_\epsilon$, the set of outputs of some processes is exactly O_{2i}, for some $i \in \{0, \ldots, \frac{k-1}{2}\}$, then no other processes output a value $d \in O_{2j}$ for any $j \neq i$ in that execution. Indeed, assume that in some execution $\mathcal{E}$, $d \in O_{2j}$ along with the values

$$d_{2i} = \frac{2i}{k}, \text{ and } d_{2i+1} = \frac{2i+1}{k}$$

are output. If $j < i$, we have

$$d_{2i+1} - d = \frac{2i+1}{k} - d \geq \frac{(2i)+1-(2(i-1)+1)}{k} = \frac{2}{k} > \epsilon.$$

Similarly, if $i < j$, then $d - d_{2i} \geq \frac{2}{k}$. Hence, in both case ϵ-agreement is not satisfied in $\mathcal{E}$.

Thanks to the setting of k, and by the pigeon hole principle, we get that there exists $i \neq j$ in $\{0, \ldots, \frac{k-1}{2}\}$ such that $w_{2i} = w_{2j}$. Let $\mathcal{E}'_{2i} = \mathcal{E}_{2i} \circ \mathcal{E}$ be an extension of $\mathcal{E}_{2i}$ in which, immediately after $\mathcal{E}_{2i}$, all processes $p_1, \ldots, p_{n-t+1}$ crash, and all the $t-1$ processes $p_{n-t+2}, \ldots, p_n$ take steps, say in a round robin fashion), until one of them decides some value d. Note that $d \in O_{2i}$ since both values $\frac{2i}{k}$ and $\frac{2i+1}{k}$ have been decided in $\mathcal{E}_{2i}$. The inputs of processes $\{p_{n-t+2}, \ldots, p_n\}$ are defined arbitrarily. As $t > \frac{n}{2}$, we have $t \geq n - t + 1$, and thus $\mathcal{E}'_{2i}$ is an execution with at most t failures. As $\mathcal{A}_\epsilon$ tolerates up to t failures, at least one of the processes

Algorithm 1 Approximate agreement protocol $\mathcal{A}_k$ for two processes. Code for process p_i, $i \in \{1, 2\}$ with input $myInput \in \{0, 1\}$

```
local variables:
    prec, new, x_1, x_2 ∈ {0,1}                    ▷ prec initialized to 0
    me, other, who ∈ {1,2} ▷ me and other resp. initialized to i and j ∈ {1,2} \ {i} at p_i
    r ∈ {1, ..., k}
1:  begin
2:      I_me.write(myInput)
3:      for r = 1, ..., k do
4:          R_me.write(r mod 2)
5:          new ← R_other.read()
6:          if new ≠ prec then prec ← new
7:          else break      ▷ leave the for-loop if the same value is read twice
8:      x_me ← I_me.read()
9:      x_other ← I_other.read()
10:     if (x_other = ⊥ or x_me = x_other) then return x_me
11:     else
12:         if (r = k and new = k mod 2) then      ▷ me left the for-loop after k iterations
13:             if r mod 2 = 0 then who ← me else who ← other
14:             return((x_who + k)/(2k + 1))
15:         else      ▷ me left the for-loop because it has read the same value twice
16:             if r mod 2 = 0 then who ← other else who ← me
17:             return(x_who + ((−1)^{x_who}(r − 1))/(2k + 1))
18: end
```

$p_{n-t+2}, \ldots, p_n$ must decide after taking finitely many steps. Since, in $\mathcal{E}_{2i}$ as well as in $\mathcal{E}_{2j}$, no processes in $\{p_{n-t+2}, \ldots, p_n\}$ take steps, and the state of the shared registers is the same at the end of both executions, the executions $\mathcal{E}_{2i}$ and $\mathcal{E}_{2j}$ are indistinguishable for the processes $p_{n-t+2}, \ldots, p_n$. The execution $\mathcal{E}'_{2j} = \mathcal{E}_{2j} \circ \mathcal{E}$ is thus a valid execution of $\mathcal{A}_\epsilon$, indistinguishable from $\mathcal{E}'_{2i}$ for processes in $\{p_{n-t+2}, \ldots, p_n\}$. It follows that d is also the output value in $\mathcal{E}'_{2j}$ for some process. Therefore, ϵ-agreement is violated in $\mathcal{E}'_{2j}$ since $d \notin O_{2j} = \{\frac{2j}{k}, \frac{2j+1}{k}\}$ and both $\frac{2j}{k}$ and $\frac{2j+1}{k}$ are outputted in the prefix $\mathcal{E}_{2j}$ of $\mathcal{E}'_{2j}$, contradicting the correctness of $\mathcal{A}_\epsilon$.

4 CONSTANT-SIZE REGISTERS ARE UNIVERSAL FOR 2-PROCESSES SYSTEMS

To establish Theorem 1.2, that is, to show that any task solvable by 2 processes in the wait-free shared-memory model with unbounded-size registers can be solved by 2 processes in the wait-free shared-memory model with 3-bits registers, we proceed in two steps. We give here an overview of these two steps of the proof. All details are in the full version of the paper[13].

(1) First, we show that, for every $\epsilon > 0$, $n = 2$ processes can solve ϵ-agreement with 1-bit registers, in Algorithm 1. Our algorithm for solving ϵ-agreement with 1-bit registers has step-complexity equal to $O(\frac{1}{\epsilon})$, which is exponentially slower than the fastest algorithm solving ϵ-agreement with unbounded registers — for which the step complexity is

$\Theta(\log \frac{1}{\epsilon})$ (see, e.g., [18]). In Section 7 we present a more involved algorithm that solves ϵ-agreement with optimal step-complexity $O(\log \frac{1}{\epsilon})$ with slightly larger constant-size registers, using ideas of Section 6.

In Algorithm 1, each of the two processes alternates writing 0 and 1 in its register, and reading the value in the other register. It stops whenever it reads twice the same value in the register of the other process. Intuitively, this allows the two processes to check whether they are running concurrently, and to stop whenever they become desynchronized, at which point they can decide output values that are close together. Which value each of the two processes adopts depends on the parity of the round at which they become desynchronized, and on which of the two processes went faster than the other. Nevertheless, the fact that the two processes are not desynchronized by more than one round allows them to adopt values close to each other. The number of rounds is limited up to $\Theta(1/\epsilon)$ read/write operations, which is enough for guaranteeing a precision of ϵ in case the two processes run concurrently in locksteps.

More formally, Algorithm 1 solves ϵ-agreement for two processes p_1 and p_2. The first step in the code is for each process $p_i \in \{p_1, p_2\}$ to write its input in the write-once special register I_i. The shared registers R_1 and R_2 are initialized to 0, and registers I_1 and I_2 to $\perp$.

A possible execution of the algorithm is given in Figure 2.

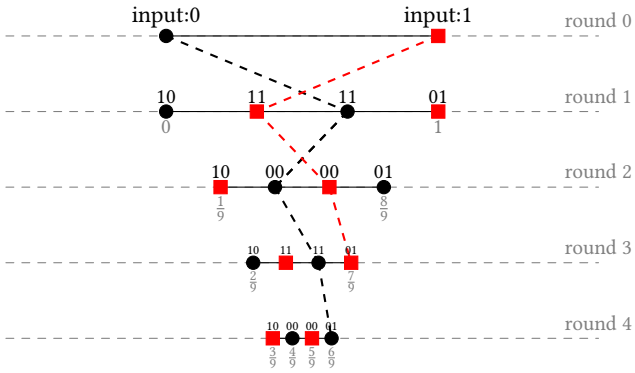


Figure 2: *Local states and an execution of Algorithm 1 with inputs* 0 *and* 1 *and* $k = 4$. *Process* p_1 *is represented by a black dot, process* p_2 *by a red square. Algorithm 1 proceeds in* asynchronous rounds. *In each round, process* $p_i, i \in \{1, 2\}$ *writes the parity of its current round to its register* R_i *and reads the other process register. Its local state is identified with its local round number (in gray, on the right side) and its view of the shared memory (in black, above each dot/square). The view consists in two bits: the one it has written in its register, and the one it reads in its current register. If the two bits differ (*p_i *is one round ahead the other),* p_i *decides a value (gray value below square/dot vertexes) depending on its index and the parity of its local round and halts. If it has not done so previously, a process can also decide at the end of round* k *(corresponding local states are the central square/dot vertexes on the bottom dashed line associated with round* $k = 4$*). In the execution pictured with dashed lines, process* p_2 *(red square) decides and halts at round 3. Process* p_1 *(black dot) decides one round later.*

(2) Second, we have to show that, in 2-processes systems with constant-size registers, a black box solving ϵ-agreement using 1-bit registers can be used for solving any wait-free solvable task. A formal proof can be found in the full version[13], here is an overview. In a nutshell, to get the intuition of why this is true, we use the classical framework introduced by Biran, Moran and Zaks [9], who gave necessary and sufficient conditions for tasks to be solvable in the 1-resilient model (which coincides with the wait-free model for $n = 2$ processes). (A similar proof can be based on Theorem 2.5.2 of [22]). At any point in time of the execution of a protocol, all possible states of the system can be described by a graph whose vertices are pairs (p, σ) where $p \in \{p_1, p_2\}$ is a process, and σ is a possible local state of p at the considered time. Two vertices are linked by an edge if the two vertices correspond to two different processes in compatible states. Starting from the input graph G_0 including all vertices (p, x) where x is a possible input value for process p, the computation evolves as a sequence of graphs $(G_t)_{t \geq 0}$, where G_t displays all possible states of the system after t steps. Assuming that all processes terminate at the same step T, an algorithm specifies an output value for each vertex of G_T, i.e., for each possible state σ of a process p after T steps. Now, all possible output states also form a graph, H including all vertices (p, y) where y is a possible output value for process p. So the algorithm maps every vertex (p, σ) of G_T to a vertex (p, y) of H. Moreover, this mapping must be a homomorphism, i.e., it must preserve the edges, as any possible global states after T steps, that is, any possible edge $\{(1, \sigma_1), (2, \sigma_2)\}$ of G_T, must be mapped to a legal global output states, i.e., to an edge $\{(1, y_1), (2, y_2)\}$ of H.

The structure of the graph G_T depends on the communication model of the system. In the case of 1-resilient computing, this graph can be viewed as a form of refinement of the input graph G_0, mostly obtained by subdividing the edges of G_0, which intuitively corresponds to capturing all possible interleaving of the read and write operations performed by the two processes (see [9]). Each process p knows in which vertex (p, σ) of G_T it stands after it has performed T steps, starting from its input (p, x) and repeating T times (1) writing its entire current history in memory, and (2) reading the current history of the other process. Here comes the use of approximate agreement. An alternative approach is to let the two processes solving ϵ-agreement, for some suited ϵ depending on T, and small enough so that there is a mapping from the output graph of ϵ-agreement to G_T. That is, given an input edge $\{(p_1, x_1), (p_2, x_2)\}$ of G_0, the two processes can ϵ-agree on an edge $\{(p_1, \sigma_1), (p_2, \sigma_2)\}$ of G_T without exchanging histories, and actually using 1-bit registers only. Once they have agreed on $\{(p_1, \sigma_1), (p_2, \sigma_2)\}$, the two processes can map the edge to a legal output configuration.

5 UNIVERSALITY WHEN A MINORITY OF PROCESSES MAY FAIL

To establish Theorem 1.3, given an algorithm $\mathcal{A}$ solving a task Π in the t-resilient shared-memory model with unbounded registers, one aims at constructing an algorithm $\mathcal{B}$ solving Π with registers of size $O(t)$ bits. This construction proceeds in three phases. Additional details can be found in the full version[13].

(1) Since $t < \frac{n}{2}$, the existence of $\mathcal{A}$ implies [4] that there exists an n-processes *message-passing* algorithm $\mathcal{A}'$ solving Π when at most t processes may crash. Recall that, in the message-passing model, instead of communicating via a shared memory, the processes communicate by exchanging messages. Every process can send a message to every other process, hence the communication network is complete. Communication channels are FIFO, and no messages are lost. However, the time required to deliver a message is unbounded. In $\mathcal{A}'$, in addition to the value being written or read, each message carries some "control bits".
(2) We show that the t-resilient message-passing model (which assumes a complete network) can be simulated by a network with fewer communication links between processes, as long as the network is $(t+1)$-connected, i.e., removing at most t nodes of the network keeps it connected. For every process p_i, when sending (or receiving) a message m to (from) some process $p_j \neq p_i$, process p_i sends m directly to p_j if there is a link (i, j), otherwise it sends it to all its $t+1$ successors. From these latter processes, the message m is forwarded using the same mechanism until it reaches its destination. As at most t processes may fail, any message sent by a non-faulty process is eventually received by its destination whenever the latter is not faulty. This results in an algorithm $\mathcal{A}''$.
(3) We rely on the *alternating bit* protocol (see [8, 29]), to simulate algorithms for the n-node $(t+1)$-connected network in the shared memory model with registers of size $O(t)$ bits. For that, we implement message passing communication on channel from p_i to p_j by a 2 bits register R_i written by p_i and read by p_j and a 1 bit register A_j (initialized to 0) written by p_j and read by p_i. For each of its $t+1$ output links p_i writes in a 2 bits register and for each of its input links it writes a 1 bit register. Regrouping this in a single register we get a $3(t+1)$ bits register. Applied to algorithm $\mathcal{A}''$, this results in the desired algorithm $\mathcal{B}$.

We note that in the construction above a process may need to keep taking step after having obtained a decision value. But tasks solvability, which our concern here, only requires every non-faulty processes decide, without forcing them to stop afterwards.

6 UNIVERSALITY OF THE ITERATED IMMEDIATE SNAPSHOT (IIS) MODEL

For proving Theorem 1.4, we show that, for any $n \geq 2$, any task solvable wait-free by n processes in the IIS model with unbounded registers is solvable wait-free in the IIS model with 1-bit registers (at each level of the memory). To establish this result, we show that *full-information* protocols, i.e., protocols using unbounded registers in which processes can write all the information acquired so far, can be emulated in the IIS model using 1-bit registers. We actually consider a weaker variant of the IIS model, where the immediate snapshots are merely replaced by *collect* operations. In the iterated collect model (IC), every memory M_r, $r \geq 1$, is read by each process using a collect() operation [7], which merely consists of reading the n registers of M_r one by one, in an arbitrary order. We then proceed in two steps:

(1) We show that any full-information protocol in the IIS model with unbounded size registers can be simulated in the IC model with unbounded size registers. For this, we simulate one round of the IIS model with n write/collect iterations using the immediate snapshot algorithm presented by Borowsky and Gafni in [10].
(2) We show that any full-information protocol solving a task in the IC model with unbounded-size registers can be simulated in the IIS model using 1-bit registers. In a sense an iterated model provides a counter to the processes "for free", corresponding to the index r of the memory M_r. We show that, with this implicit counter, 1-bit registers suffice for the model to be universal. Intuitively, instead of writing a large value v in memory, a process can merely write 1 in its register in M_v, and write 0 in all the other registers. This approach is over simplified, and does not suffice for simulating the IIS model using 1-bit registers. Nevertheless, this provides a hint on how one can play with the level r of the memory M_r to encode large values.

All details of the proof can be found in the full version[13].

7 FASTER 2-PROCESSES WAIT-FREE ϵ-AGREEMENT

A key building block for establishing the universality of constant size registers for two processes is a binary ϵ-agreement protocol with 1-bit registers (Algorithm 1). When there is no restriction on the size of the registers, 2-processes wait-free binary ϵ-agreement is known to be solvable in $O(\log \frac{1}{\epsilon})$ steps per process, and this bound is best possible (see, e.g. [25]). Algorithm 1 uses registers of size 1 bit only, but is exponentially slower, with $O(\frac{1}{\epsilon})$ steps per process. In this section, we show that this slowdown is not inherent to the fact that each register has constant size. We present a more involved solution performing in $O(\log \frac{1}{\epsilon})$ steps, for two processes using registers of constant size.

The final states of a protocol $\mathcal{P}$ can be arranged in a graph $G(\mathcal{P})$, called the *protocol complex* in the literature since [23]. Each vertex is a final local state of some process, and an edge connects two vertexes whenever the corresponding local states coexist at the end of some execution of the protocol. An important fact about wait-free models is that the protocol graph is connected. In the iterated immediate snapshot model IIS, for two processes and fixed inputs, it is known [22, chapter 2] that the graph is a path. This is in particular the case of ϵ-agreement protocols, where the input of process p_1 is 0 and the input of process p_2 is 1. Each extremity corresponds to a *solo* execution, in which one process completes the protocol without seeing a step by the other process. The ability of a protocol to solve ϵ-agreement depends on the length of the path: the longer the path is, the smaller ϵ can be chosen. Indeed,

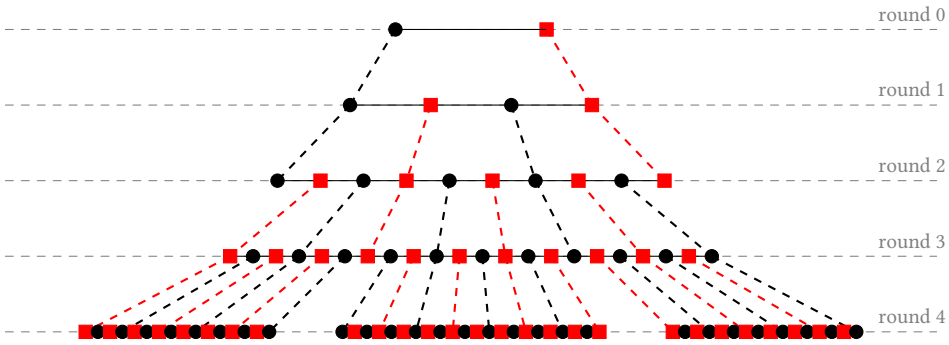


Figure 3: *Simulated subset of IIS executions. Each process exits the simulation after simulating $\Delta = 2$ consecutive solo-round (dashed-lines.) or after simulating $R = 4$ rounds. The number of simulated IIS executions still grows exponentially with R.*

in a solo execution process p with input $x \in \{0, 1\}$ has to decide x and therefore ϵ cannot be smaller than $\frac{1}{L}$, where L is the length of the path.

How fast does the length of the path grow as processes take more and more steps? In the IIS model for two processes, with no restriction on the size of the registers in each memory M_r, the length is 3^R after R rounds. Binary ϵ-agreement can be solved in R rounds, for any $\epsilon \geq \frac{1}{3^R}$ by assigning the values $0, \frac{1}{3^R}, \frac{2}{3^R}, \ldots, 1$ to the vertexes representing the local states after R rounds, starting from the vertex representing the state of the process with input 0 running solo. The difficulty of restricting the size of the registers is that the path after R rounds may be shorter, due to the inability to distinguish executions, which in a full-information protocol would be distinguishable. Perhaps surprisingly, it was shown in [14] that each process writing only one bit of information in each round is sufficient for the set of local states after R rounds to still be of size 3^R after R rounds. It follows that binary ϵ-agreement can be solved in $O(\log \frac{1}{\epsilon})$ rounds by two processes in the IIS model with 2-bits registers.

R-rounds IIS executions can be simulated in a wait-free manner using SWMR registers of size at least $\Omega(R)$ [10, 11]. Linear space in the number of simulated rounds is required, as one process p may be simulating some round r while the other process p' may still be simulating some lower round r'. The difference $r - r'$ cannot be bounded below R, and enough information has to be kept in shared memory for p' to be able to simulate rounds $r' + 1, \ldots, r$. This is because p' cannot ask and wait for p to tell him the values it has written in rounds $r' + 1, \ldots, r$. We get around this issue by simulating only a *subset* of all possible R-round IIS executions in a *non-uniform* way. Essentially, after a process has simulated Δ consecutive solo rounds, it quits the simulation, where Δ is a parameter of the simulation. This allows to restrict the size of the SWMR registers to $O(\Delta \times \log |\mathcal{V}|)$ bits, since a process has only to make available the values it has written in the last Δ rounds of the simulated IIS execution ($\mathcal{V}$ is the set of values written by the simulated IIS protocol).

Of course some IIS executions are never simulated (namely, the ones in which a process is solo for more than Δ rounds). Nevertheless, for any $R > 0$, we show that the total number (over all executions of the simulation protocol) of simulated IIS executions of at most R rounds is $\Omega(2^R)$ for $\Delta \geq 2$. Figure 3 shows the subset of IIS executions for the case $R = 4$ and $\Delta = 2$.

We fix $\Delta = 2$ and simulate IIS executions of at most R rounds of the IIS protocol in [14] that communicate only a constant number of bits per round, while keeping the number of distinct final local states as high as in a full information protocol. The number of distinct local states, over all executions of the simulation protocol, is $\Omega(2^R)$, from which ϵ-agreement with $\epsilon \geq \frac{1}{2^R}$ can be solved. Each process register is thus of constant size (since Δ is fixed independently of ϵ and the simulated IIS protocol communicate a constant number of bits per round). As for two processes, simulating one IIS round takes a constant number of writes and reads, the step complexity is $O(R) = O(\log \frac{1}{\epsilon})$, as desired. Detailed explanations on the construction as well as the pseudo-code of the simulation can be found in the full version of the paper [13].

8 CONCLUSION

In this paper, we have studied the standard read/write shared-memory model where n asynchronous crash-prone processes communicate using one single-writer/multi-reader register per process. For the case where the maximum number t of crash failures satisfies $1 \leq t < \frac{n}{2}$, we have showed that registers of size $O(t)$ are sufficient. So, for $t = O(1)$, the t-resilient model with constant-size registers is universal. We have given special attention to the case of $n = 2$, where we have discovered interesting new ways of solving approximate agreement, the basic building block to solve any task. We have shown that, for a system of two processes, 3-bit registers are universal. We have designed a novel 2-processes approximate agreement algorithm with 1-bit registers, and an optimal time, more complex version, using 6-bits registers. On the other hand, when a majority of the processes can crash, the size of the registers must depend not only on the number n of processes, but also on the task itself.

Implications. In general, given two non-decreasing functions $\rho, \sigma : \mathbb{N} \to \mathbb{N}$, one may ask what tasks can be computed by n processes communicating with $\rho(n)$ registers, each of size $\sigma(n)$. Our results imply that for $t > \frac{n}{2}$ and any ρ, σ, there are tasks that cannot be solved with $\rho(n)$ registers of size $\sigma(n)$ but can be solved t-resiliently using n unbounded single-writer/multi-reader registers. This follows from Theorem 1.1 and the fact that one (large, but

still bounded) SWMR register may be used to simulate a bounded number of SWMR registers (by dividing the space the register among the simulated ones.). Furthermore, this extends to the case where the bounded registers can be written by multiple processes, as bounded size multi-writer/multi-reader registers can simulated with n bounded size SWMR registers (construction can be found in standard textbooks, e.g.[24].). For constant size registers, our results on the universality of the 1 bit-(Immediate) Iterated Snapshot model imply that a shared-memory consisting in an unbounded number of SWMR binary registers (i.e., $\sigma(n) = 1$) is universal. Indeed, to emulate the 1-bit IS model, one can arrange the infinite supply of binary registers in an infinite sequence $M_1, \ldots, M_r, \ldots$ of arrays of n registers. Snapshot in iteration r can be obtained by performing a successful double collect of M_r. This always terminates as each register of M_r is modified at most once.

Open problems. Several interesting questions are left for future work, including resolving the case of $t = n/2$. Perhaps the most significant one is finding a characterization (topological or by other means) of the tasks that are wait-free solvable in the read/write shared-memory model with constant-size registers, or with registers of $f(n)$ bits for some given function f.

The intimate relation between task computability and topology has been thoroughly investigated, using unbounded size registers, to be able to define a canonical, full-information protocol. Namely, a task is solvable if and only if it is solvable by a full-information protocol. This allows to use a simplicial complex that represents all possible executions of a full-information protocol, whose topology determines task solvability. When the registers are of bounded size, is it possible to define a canonical protocol? What is the effect on the topology of limiting the size of the registers? Our results for the case of $n = 2$ give a hint of how interesting these questions can be.

Similar questions can be asked for other models of distributed computing, including for example shared objects more powerful than read/write registers. We stress that the focus of this paper was on computability, we leave for future work to study how restricting the size of the registers affects step complexity.

ACKNOWLEDGMENTS

We thank the anonymous reviewers whose comments help improve the presentation of the paper. This work is funded in part by the ANR project DUCAT ANR-20-CE48-0006.

REFERENCES

[1] Ittai Abraham, Yonatan Amit, and Danny Dolev. 2004. Optimal Resilience Asynchronous Approximate Agreement. In *8th Int. Conference on Principles of Distributed Systems (OPODIS) (LNCS, Vol. 3544)*. Springer, 229–239. https://doi.org/10.1007/11516798_17

[2] Yehuda Afek, Hagit Attiya, Danny Dolev, Eli Gafni, Michael Merritt, and Nir Shavit. 1993. Atomic Snapshots of Shared Memory. *J. ACM* 40, 4 (1993), 873–890. https://doi.org/10.1145/153724.153741

[3] Yehuda Afek and Eli Gafni. 2015. A simple characterization of asynchronous computations. *Theor. Comput. Sci.* 561 (2015), 88–95. https://doi.org/10.1016/j.tcs.2014.07.022

[4] Hagit Attiya, Amotz Bar-Noy, and Danny Dolev. 1995. Sharing Memory Robustly in Message-Passing Systems. *J. ACM* 42, 1 (1995), 124–142. https://doi.org/10.1145/200836.200869

[5] Hagit Attiya and Faith Ellen. 2022. The Step Complexity of Multidimensional Approximate Agreement. In *26th Int. Conference on Principles of Distributed Systems (OPODIS) (LIPIcs, Vol. 253)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 6:1–6:12. https://doi.org/10.4230/LIPIcs.OPODIS.2022.6

[6] Hagit Attiya and Sergio Rajsbaum. 2002. The Combinatorial Structure of Wait-Free Solvable Tasks. *SIAM J. Comput.* 31, 4 (2002), 1286–1313. https://doi.org/10.1137/S0097539797330689

[7] Hagit Attiya and Jennifer Welch. 2004. *Distributed computing: fundamentals, simulations, and advanced topics*. Vol. 19. John Wiley & Sons.

[8] Keith A. Bartlett, Roger A. Scantlebury, and Peter T. Wilkinson. 1969. A note on reliable full-duplex transmission over half-duplex links. *Commun. ACM* 12, 5 (1969), 260–261.

[9] Ofer Biran, Shlomo Moran, and Shmuel Zaks. 1990. A Combinatorial Characterization of the Distributed 1-Solvable Tasks. *J. Algorithms* 11, 3 (1990), 420–440. https://doi.org/10.1016/0196-6774(90)90020-F

[10] Elizabeth Borowsky and Eli Gafni. 1993. Immediate Atomic Snapshots and Fast Renaming (Extended Abstract). In *12th ACM Symposium on Principles of Distributed Computing (PODC)*. ACM, 41–51. https://doi.org/10.1145/164051.164056

[11] Elizabeth Borowsky and Eli Gafni. 1997. A Simple Algorithmically Reasoned Characterization of Wait-Free Computations (Extended Abstract). In *16th ACM Symposium on Principles of Distributed Computing (PODC)*. 189–198. https://doi.org/10.1145/259380.259439

[12] Armando Castañeda, Sergio Rajsbaum, and Michel Raynal. 2011. The renaming problem in shared memory systems: An introduction. *Comput. Sci. Rev.* 5, 3 (2011), 229–251. https://doi.org/10.1016/j.cosrev.2011.04.001

[13] Carole Delporte, Hugues Fauconnier, Pierre Fraigniaud, Sergio Rajsbaum, and Corentin Travers. 2023. The Computational Power of Distributed Shared-Memory Models with Bounded-Size Registers. *CoRR* abs/2309.13977 (2023). https://doi.org/10.48550/ARXIV.2309.13977 arXiv:2309.13977

[14] Carole Delporte-Gallet, Hugues Fauconnier, and Sergio Rajsbaum. 2020. Communication Complexity of Wait-Free Computability in Dynamic Networks. In *27th Int. Colloquium on Structural Information and Communication Complexity (SIROCCO) (LNCS, Vol. 12156)*. Springer, 291–309. https://doi.org/10.1007/978-3-030-54921-3_17

[15] Danny Dolev, Nancy A. Lynch, Shlomit S. Pinter, Eugene W. Stark, and William E. Weihl. 1983. Reaching Approximate Agreement in the Presence of Faults. In *3rd IEEE Symp. on Reliability in Distributed Software and Database Systems (SRDS)*. 145–154.

[16] Danny Dolev, Nancy A. Lynch, Shlomit S. Pinter, Eugene W. Stark, and William E. Weihl. 1986. Reaching approximate agreement in the presence of faults. *J. ACM* 33, 3 (1986), 499–516. https://doi.org/10.1145/5925.5931

[17] Michael J. Fischer, Nancy A. Lynch, and Mike Paterson. 1985. Impossibility of Distributed Consensus with One Faulty Process. *J. ACM* 32, 2 (1985), 374–382. https://doi.org/10.1145/3149.214121

[18] Matthias Függer, Thomas Nowak, and Manfred Schwarz. 2021. Tight Bounds for Asymptotic and Approximate Consensus. *J. ACM* 68, 6 (2021), 46:1–46:35. https://doi.org/10.1145/3485242

[19] Eli Gafni and Sergio Rajsbaum. 2010. Distributed Programming with Tasks. In *14th Int. Conference on Principles of Distributed Systems (OPODIS) (LNCS, Vol. 6490)*. Springer, 205–218. https://doi.org/10.1007/978-3-642-17653-1_17

[20] Emmanuel Godard and Eloi Perdereau. 2020. Back to the Coordinated Attack Problem. *Math. Struct. Comput. Sci.* 30, 10 (2020), 1089–1113. https://doi.org/10.1017/S0960129521000037

[21] Maurice Herlihy. 1991. Wait-Free Synchronization. *ACM Trans. Program. Lang. Syst.* 13, 1 (1991), 124–149. https://doi.org/10.1145/114005.102808

[22] Maurice Herlihy, Dmitry N. Kozlov, and Sergio Rajsbaum. 2013. *Distributed Computing Through Combinatorial Topology*. Morgan Kaufmann.

[23] Maurice Herlihy and Nir Shavit. 1999. The topological structure of asynchronous computability. *J. ACM* 46, 6 (1999), 858–923. https://doi.org/10.1145/331524.331529

[24] Maurice Herlihy and Nir Shavit. 2008. *The art of multiprocessor programming*. Morgan Kaufmann.

[25] Gunnar Hoest and Nir Shavit. 2006. Toward a Topological Characterization of Asynchronous Complexity. *SIAM J. Comput.* 36, 2 (2006), 457–497. https://doi.org/10.1137/S0097539701397412

[26] Damien Imbs, Sergio Rajsbaum, and Adrián Valle. 2015. Untangling Partial Agreement: Iterated x-consensus Simulations. In *17th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS) (LNCS, Vol. 9212)*. Springer, 139–155. https://doi.org/10.1007/978-3-319-21741-3_10

[27] Petr Kuznetsov and Thibault Rieutord. 2020. Affine Tasks for k-Test-and-Set. In *22nd International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS) (LNCS, Vol. 12514)*. Springer, 151–166. https://doi.org/10.1007/978-3-030-64348-5_12

[28] Petr Kuznetsov, Thibault Rieutord, and Yuan He. 2018. An Asynchronous Computability Theorem for Fair Adversaries. In *37th ACM Symposium on Principles of Distributed Computing (PODC)*. ACM, 387–396. https://dl.acm.org/citation.cfm?id=3212765

[29] William C. Lynch. 1968. Computer systems: Reliable full-duplex file transmission over half-duplex telephone line. *Commun. ACM* 11, 6 (1968), 407–410.

[30] Shlomo Moran. 1995. Using Approximate Agreement to Obtain Complete Disagreement: The Output Structure of Input-Free Asynchronous Computations. In *3rd Israel Symposium on Theory of Computing and Systems (ISTCS)*. IEEE, 251–257. https://doi.org/10.1109/ISTCS.1995.377025

[31] Alon Orlitsky and Krishnamurthy Viswanathan. 2003. One-way communication and error-correcting codes. *IEEE Trans. Inf. Theory* 49, 7 (2003), 1781–1788. https://doi.org/10.1109/TIT.2003.813496

[32] Sergio Rajsbaum. 2023. A distributed computing perspective of unconditionally secure information transmission in Russian cards problems. *Theor. Comput. Sci.* 952 (2023), 113761. https://doi.org/10.1016/j.tcs.2023.113761

[33] Michel Raynal. 2013. *Concurrent Programming - Algorithms, Principles, and Foundations*. Springer. https://doi.org/10.1007/978-3-642-32027-9

[34] Michel Raynal. 2016. Set Agreement. In *Encyclopedia of Algorithms*. Springer, 1956–1959. https://doi.org/10.1007/978-1-4939-2864-4_367

[35] Vikram Saraph, Maurice Herlihy, and Eli Gafni. 2016. Asynchronous Computability Theorems for t-Resilient Systems. In *30th Int. Symposium on Distributed Computing (DISC) (Lecture Notes in Computer Science, Vol. 9888)*. Springer, 428–441. https://doi.org/10.1007/978-3-662-53426-7_31

[36] Eric Schenk. 1995. Faster Approximate Agreement with Multi-Writer Registers. In *36th IEEE Symposium on Foundations of Computer Science (FOCS)*. IEEE Computer Society Press, 714–723. https://doi.org/10.1109/SFCS.1995.492673

[37] Eric Schenk. 1996. *Computability and complexity results for agreement problems in shared memory distributed systems*. Ph. D. Dissertation. University of Toronto, Canada. https://librarysearch.library.utoronto.ca/permalink/01UTORONTO_INST/14bjeso/alma991106419166406196